```
const obj = { a: 1 };

console.log(obj.a);
```

2

```
const obj = { a: 1 };

console.log(obj["a"]);
```

3

```
const key = "a";

const obj = { a: 10 };

console.log(obj[key]);
```

4

```
const obj = {};

obj.a = 5;

obj.a = 10;

console.log(obj.a);
```

5

```
const obj = { a: 1 };

const ref = obj;

ref.a = 5;
```

```
console.log(obj.a);
```

6

```
const obj1 = { a: 1 };
const obj2 = { a: 1 };
console.log(obj1 === obj2);
```

7

```
const obj = { a: 1 };
console.log("a" in obj);
```

8

```
const obj = { a: 1 };
console.log(obj.hasOwnProperty("a"));
```

9

```
const obj = { a: 1 };
delete obj.a;
console.log(obj.a);
```

10

```
const obj = { a: 1 };
console.log(Object.keys(obj));
```

11

```
const obj = { a: 1 };

console.log(Object.values(obj));
```

12

```
const obj = { a: 1 };

console.log(Object.entries(obj));
```

13

```
const obj = { a: undefined };

console.log(obj.hasOwnProperty("a"));
```

14

```
const obj = {};

console.log(obj.toString());
```

15

```
const obj = { a: 1 };

console.log(typeof obj);
```

🟡 Intermediate (16–35)

16

```
const obj = { a: 1 };
```

```
const newObj = obj;

newObj.a = 100;

console.log(obj.a);
```

17

```
const obj = { a: 1 };

const newObj = { ...obj };

newObj.a = 100;

console.log(obj.a);
```

18

```
const obj = { a: { b: 1 } };

const newObj = { ...obj };

newObj.a.b = 100;

console.log(obj.a.b);
```

19

```
const obj = { a: 1 };

Object.freeze(obj);

obj.a = 100;

console.log(obj.a);
```

20

```
const obj = { a: { b: 1 } };
Object.freeze(obj);
obj.a.b = 100;
console.log(obj.a.b);
```

21

```
const obj = { a: 1 };
Object.seal(obj);
obj.a = 5;
console.log(obj.a);
```

22

```
const obj = { a: 1 };
Object.seal(obj);
obj.b = 2;
console.log(obj.b);
```

23

```
const obj = { a: 1 };
Object.defineProperty(obj, "a", { writable: false });
```

```
obj.a = 10;

console.log(obj.a);
```

24

```
const obj = { a: 1 };

Object.defineProperty(obj, "b", { value: 2 });

console.log(obj.b);
```

25

```
const obj = { a: 1 };

Object.defineProperty(obj, "b", { value: 2, enumerable: false
});

console.log(Object.keys(obj));
```

26

```
const obj = { a: 1 };

for (let key in obj) {

 console.log(key);

}
```

27

```
const obj = Object.create({ a: 1 });

obj.b = 2;
```

```
console.log(obj.a);
```

28

```
const obj = Object.create(null);

console.log(obj.toString);
```

29

```
const obj = { a: 1 };

console.log(obj.__proto__ === Object.prototype);
```

30

```
const obj = {};

console.log(obj.__proto__ === Object.prototype);
```

31

```
const obj = { a: 1 };

console.log(obj.propertyIsEnumerable("a"));
```

32

```
const obj = {};

Object.defineProperty(obj, "a", { value: 1, enumerable: false
});

console.log(obj.propertyIsEnumerable("a"));
```

33

```
const obj = { a: 1 };

console.log(Object.getOwnPropertyNames(obj));
```

34

```
const obj = { a: 1 };

Object.preventExtensions(obj);

obj.b = 2;

console.log(obj.b);
```

35

```
const obj = { a: 1 };

Object.preventExtensions(obj);

delete obj.a;

console.log(obj.a);
```

🔴 Advanced / Tricky (36–50)

36

```
const obj = {

 a: 1,

 getA() {
```

```
  return this.a;
 }
};
const fn = obj.getA;
console.log(fn());
```

37

```
const obj = {
 a: 1,
 getA: function() {
  return this.a;
 }
};
const fn = obj.getA;
console.log(fn());
```

38

```
const obj = {
 a: 1,
 getA: () => this.a
```

```
};
console.log(obj.getA());
```

39

```
const obj = {
 a: 1,
 getA() {
   return this.a;
 }
};
console.log(obj.getA.call({ a: 100 }));
```

40

```
const obj = {
 a: 1
};
function test() {
 console.log(this.a);
}
test.call(obj);
```

41

```
const obj = { a: 1 };
function test() {
 return this.a;
}
const bound = test.bind(obj);
console.log(bound());
```

42

```
const obj = { a: 1 };
function test() {
 return this.a;
}
const bound = test.bind(obj);
obj.a = 100;
console.log(bound());
```

43

```
const obj = { a: 1 };
const newObj = JSON.parse(JSON.stringify(obj));
```

```
newObj.a = 100;

console.log(obj.a);
```

44

```
const obj = { a: undefined };

console.log(JSON.stringify(obj));
```

45

```
const obj = { a: function() {} };

console.log(JSON.stringify(obj));
```

46

```
const obj = { a: 1 };

const clone = structuredClone(obj);

clone.a = 5;

console.log(obj.a);
```

47

```
const obj = { a: { b: 1 } };

const clone = structuredClone(obj);

clone.a.b = 100;

console.log(obj.a.b);
```

48

```
const obj = {};
Object.defineProperty(obj, "a", {
 get() {
   return 5;
 }
});
console.log(obj.a);
```

49

```
const obj = {
 a: 1,
 b: 2
};
const { a, ...rest } = obj;
console.log(rest);
```

50

```
const obj = { a: 1 };
const newObj = Object.assign({}, obj);
```

```
newObj.a = 10;

console.log(obj.a);

const obj = { a: 1 };

Object.seal(obj);

obj.a = 5;

delete obj.a;


console.log(obj.a);
```

2

```
const obj = { a: 1 };

Object.seal(obj);


obj.b = 2;


console.log(obj.b);
```

3

```
const obj = { a: { b: 1 } };
Object.seal(obj);

delete obj.a;

console.log(obj.a.b);
```

4

```
const obj = { a: 1 };
Object.seal(obj);

Object.defineProperty(obj, "a", { writable: false });

obj.a = 10;

console.log(obj.a);
```

5

```
const obj = { a: 1 };

Object.seal(obj);


Object.defineProperty(obj, "b", { value: 2 });


console.log(obj.b);
```

6

```
const obj = { a: 1 };

Object.seal(obj);


Object.defineProperty(obj, "a", { configurable: true });
```

👉 What happens here?

7

```
const obj = { a: 1 };
Object.seal(obj);

console.log(Object.isSealed(obj));
```

8

```
const obj = { a: 1 };
Object.seal(obj);

console.log(Object.isFrozen(obj));
```

9

```
const obj = { a: 1 };
Object.seal(obj);

Object.preventExtensions(obj);
```

```
obj.a = 100;

console.log(obj.a);
```

---

10 🔥 (VERY TRICKY)

```
const obj = { a: 1 };

Object.seal(obj);

const newObj = { ...obj };

newObj.a = 100;

console.log(obj.a);
```

`this` + function reference

```
const obj = {
 a: 10,
 getA() {
   return this.a;
 }
```

```
};

const fn = obj.getA;
console.log(fn());
```

---

2 🔥 Arrow vs normal function

```
const obj = {
 a: 10,
 getA: () => {
   return this.a;
 }
};

console.log(obj.getA());
```

---

3 🔥 Method chaining + this

```
const obj = {
 a: 1,
 inc() {
   this.a++;
   return this;
 }
};

console.log(obj.inc().inc().a);
```

---

4 🔥 Shallow copy trap

```
const obj = { a: { b: 1 } };
const clone = { ...obj };

clone.a.b = 100;

console.log(obj.a.b);
```

---

5 🔥 JSON clone trap

```
const obj = {
 a: 1,
 b: undefined,
 c: function () {}
```

```
};

const clone = JSON.parse(JSON.stringify(obj));

console.log(clone);
```

---

6 🔥 Key coercion (VERY IMPORTANT)

```
const obj = {};

const key1 = {};
const key2 = {};

obj[key1] = "value1";
obj[key2] = "value2";

console.log(obj[key1]);
```

---

7 🔥 Prototype chain lookup

```
const parent = { a: 1 };
const child = Object.create(parent);

child.a = 10;

console.log(child.a);
delete child.a;
console.log(child.a);
```

---

8 🔥 Object.defineProperty trap

```
const obj = {};

Object.defineProperty(obj, "a", {
 value: 10,
 writable: false
});

obj.a = 50;

console.log(obj.a);
```

---

9 🔥 Destructuring + rest

```
const obj = { a: 1, b: 2, c: 3 };

const { a, ...rest } = obj;

rest.b = 100;

console.log(obj.b);
```

---

10 🔥 bind vs call (VERY IMPORTANT)

```
const obj = { a: 1 };

function test() {
 return this.a;
}

const bound = test.bind(obj);

obj.a = 100;

console.log(bound());
```

1 🔥 `this` + nested function

```
const obj = {
 a: 10,
 getA() {
   function inner() {
     return this.a;
   }
   return inner();
 }
};

console.log(obj.getA());
```

---

2 🔥 Fixing `this` with arrow

```
const obj = {
 a: 10,
 getA() {
   const inner = () => this.a;
   return inner();
 }
};

console.log(obj.getA());
```

---

3 🔥 `this` + setTimeout

```
const obj = {
 a: 10,
 getA() {
   setTimeout(function () {
     console.log(this.a);
   }, 0);
 }
};

obj.getA();
```

---

4 🔥 `this` + arrow in setTimeout

```
const obj = {
 a: 10,
 getA() {
   setTimeout(() => {
     console.log(this.a);
   }, 0);
 }
};

obj.getA();
```

---

5 🔥 Object mutation vs reassignment

```
let obj = { a: 1 };

function test(o) {
 o.a = 100;
 o = { a: 200 };
}
```

```
test(obj);

console.log(obj.a);
```

---

6 🔥 Prototype + hasOwnProperty

```
const parent = { a: 1 };
const child = Object.create(parent);

child.a = 2;

console.log(child.hasOwnProperty("a"));
delete child.a;
console.log(child.hasOwnProperty("a"));
console.log(child.a);
```

---

7 🔥 `Object.assign` vs nested object

```
const obj = { a: { b: 1 } };

const clone = Object.assign({}, obj);

clone.a.b = 100;

console.log(obj.a.b);
```

---

8 🔥 Getter behavior

```
const obj = {
 get a() {
   return 5;
 }
};

console.log(obj.a);
obj.a = 100;
console.log(obj.a);
```

---

9 🔥 `Object.freeze` trap

```
const obj = { a: { b: 1 } };
```

```
Object.freeze(obj);

obj.a.b = 100;

console.log(obj.a.b);
```

---

10 🔥 Destructuring default + undefined

```
const obj = { a: undefined };

const { a = 10 } = obj;

console.log(a);
```

🎯 Answer Format

1 → ?
2 → ?
...
10 → ?